

Put your service behind HTTPS

Session 6 – TLS, data in transit & Wireshark

Exercise 6 · Session 6 lab · Estimated time: 80–110 min

Write your answers in the answer document `SKY2100_Exercise6_Answers.docx`. Last week you ran Nextcloud over plain HTTP. Today you *see* why that is dangerous, then fix it with TLS. You will capture your own login in the clear, turn on HTTPS, and capture again to prove the traffic is now unreadable.

Caution: Capture only your *own* traffic in your *own* VM. Sniffing other people's traffic is both unethical and illegal. This lab is about understanding, not interception.

1 Capture the problem (plain HTTP)

- ▷ Make sure Nextcloud from Exercise 5 is reachable over HTTP (e.g. `http://localhost:8080`).
- ▷ Start a capture on the loopback (or VM) interface and log in once:

```
sudo tcpdump -i lo -s 0 'tcp port 8080' -w http.pcap
```

Open the capture in Wireshark (or `tcpdump -A -r http.pcap`) and find the login request.

- ▷ What can you read in the captured request? Quote the field names you find (do *not* write a real password here).

2 Turn on TLS (HTTPS)

- ▷ Put Nextcloud behind a reverse proxy with TLS.

The quickest option is Caddy, which obtains/serves a certificate automatically; a self-signed certificate via Nginx is also fine. Example with Caddy:

```
sudo apt install -y caddy
sudo caddy reverse-proxy -from https://localhost -to localhost:8080
```

Browse to `https://localhost` and accept the certificate (self-signed in the lab).

3 Capture again (HTTPS) and compare

- ▷ Repeat the capture while logging in over HTTPS:

```
sudo tcpdump -i lo -s 0 'tcp port 443' -w https.pcap
```

- ▷ Compare the two captures. Can you find the credentials this time? What do you see instead?
- ▷ In Wireshark, inspect the TLS handshake. Note the **TLS version** and that a **certificate** is exchanged.

Note: Capture `tcp port 8080` once more during the HTTPS login: the hop from Caddy to Nextcloud is still plain HTTP inside your VM. TLS ends at the proxy — keep this in mind for the Azure reflection below.

Key idea: Put the two captures side by side. It is the same login, but in the first the field names and the value are plain text anyone on the path could read, and in the second there is nothing but ciphertext. Only one thing changed, TLS, and it turned readable traffic into noise. Notice what TLS does not do. It protects the data while it travels, but the server still decrypts and reads your password at the other end, and the capture still shows which host you talked to and when. Encryption in transit is one specific guarantee, not a blanket of secrecy, which is why later sessions still deal with access control and with protecting data at rest.

4 Cloud transfer: Microsoft Learn

▷ Work through “Implement Azure Key Vault”

and note what a vault stores.

learn.microsoft.com/training/modules/implement-azure-key-vault

▷ Name the three kinds of object a Key Vault holds, and which one a TLS certificate is.

5 Azure reflection

▷ In Azure, TLS is often terminated at an Application Gateway using a certificate from Key Vault. Name one advantage of terminating TLS at the gateway, and one risk it introduces for traffic *behind* the gateway.

6 Knowledge check

1. What three guarantees does TLS provide for data in transit?
Hint: Secrecy, tamper-detection, and knowing who the server really is.
2. Difference between **symmetric** and **asymmetric** encryption (one line each).
Hint: One shared key for both directions, versus a public/private pair.
3. Why does TLS use *both* (hybrid encryption)?
Hint: Asymmetric once to agree a key, then fast symmetric for the actual data.
4. What is a **hash** used for, and why is it not encryption?
Hint: A one-way fingerprint — it hides nothing and cannot be reversed.
5. What does a **certificate** bind together, and who signs it?
Hint: A public key to an identity (a hostname), signed by a Certificate Authority.
6. Explain the **chain of trust** in one or two sentences.
Hint: Server certificate to an intermediate CA to a root the browser already trusts.
7. What attack does certificate *validation* prevent, and how?
Hint: Man-in-the-middle — an attacker cannot get a valid cert for a domain it does not own.
8. Self-signed vs CA-signed: when is each appropriate?
Hint: Self-signed for labs and internal testing; CA-signed for anything users reach.
9. Name two **common TLS mistakes**.
Hint: Expired certs, obsolete protocols, weak ciphers, mixed content.
10. What does **forward secrecy** protect against?
Hint: It protects yesterday's recorded traffic even if the key leaks tomorrow.
11. Data in transit vs data at rest: give the typical control for each (CSA Domain 9).
Hint: TLS in transit; disk or storage encryption at rest.
12. (Reflection) TLS hid your password on the wire. Name one thing TLS does *not* protect.

Progress check: Discuss your answers in the group, then talk the teacher through them verbally at the next session only to track progress; nothing is handed in, signed or graded.

Solutions

Work through the lab first, then check here. Model answers are indicative — equivalent reasoning is fine.

Note: Model answers are indicative – accept equivalent reasoning. Multiple-choice answers are in **bold**.

Knowledge check

1. **Confidentiality** (encryption), **integrity** (hashing / MAC), and **authenticity** (the server's certificate).
2. **Symmetric** uses one shared key for both encrypt and decrypt – fast, but with a key- distribution problem. **Asymmetric** uses a public/private key pair – it solves distribution but is slower.
3. TLS uses asymmetric crypto once, to authenticate the server and agree on a shared key, then fast symmetric encryption for the data – the best of both (hybrid).
4. A **hash** is a one-way fingerprint used to *detect change* (integrity). It is not encryption: it hides nothing and cannot be reversed.
5. A certificate binds a **public key** to an **identity** (e.g. a hostname), **signed by a Certificate Authority**.
6. The server certificate is signed by an intermediate CA, which is signed by a root CA trusted by the browser. If every signature checks out to a trusted root, the certificate is trusted (PKI).
7. **Man-in-the-middle**. An attacker cannot present a CA-signed certificate for a domain it does not own, so a validating client detects the imposter.
8. **Self-signed** – labs / internal testing (encrypts, but browsers warn; no third-party identity). **CA-signed** – anything public (accepted silently).
9. Any two of: expired certificates; obsolete protocols (SSLv3, TLS 1.0/1.1) or weak ciphers; mixed content; missing HSTS; a leaked private key.
10. **Forward secrecy** protects *past* recorded sessions even if the server's long-term private key is later compromised (each session uses an ephemeral key).
11. In transit → **TLS**; at rest → **storage/disk encryption** with careful key management. (CSA Domain 9.)
12. Reflection – TLS does not protect: metadata (that you connected and to which host); data *after* TLS terminates; a compromised endpoint; or a malicious site that holds a valid certificate.

Lab checkpoints

- **HTTP capture:** the POST body shows the credentials in cleartext (`user=...&password=...`). This is the “aha” moment of the lab.
- **HTTPS capture:** only the TLS handshake (ClientHello, ServerHello, Certificate) is visible; the application data is encrypted (“Application Data” records) – no readable password. The TLS version should be **1.2 or 1.3**.
- A self-signed certificate is expected to trigger a browser warning – a good prompt to discuss *why* validation matters.

References

CSA Security Guidance v5, **Domain 9: Data Security** (verified); Jøsang – cryptography chapter (syllabus, page numbers not verified); Comer Ch. 17 – Cloud Security & Privacy (p. 233–243, context, verified sections). MS Learn: *Implement Azure Key Vault* (verified, June 2026) – a vault stores **keys**,

secrets and **certificates**; a TLS certificate is a *certificate* object.